Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică
Program: Software Engineering

Operating Systems

Laboratory Work No. 2

Meet the OS You Will Build

Track B: Building and Exploring xv6

Done by:
st. gr. FAF-241
Serghei Barladean

Checked by:
Patricia Reitman

Chișinău – 2026

Contents

1. Introduction....3
2. Building and Running xv6....3
3. Using xv6 as the Unix It Is....4
3.1 Observations....4
3.2 Example Command Output....4
4. Reading the Source....6
4.1 System Calls Used by cat....6
4.2 Kernel Implementation....6
4.3 Kernel and User Space....7
5. Creating a User Program....7
5.1 Testing....8
6. Conclusion....8

1. Introduction

The purpose of this laboratory work is to become familiar with xv6, a small Unix-like teaching operating system.

During the laboratory work, xv6 was built and launched using QEMU, basic Unix commands were executed inside the xv6 shell, and the source code was inspected in order to understand the difference between user programs and kernel code.

The laboratory work also demonstrated how user programs interact with the kernel through system calls.

2. Building and Running xv6

The experiment was performed on Ubuntu 24.04 running in a Multipass virtual machine on macOS (Apple Silicon, aarch64). The xv6 source code was cloned from the official mit-pdos/xv6-riscv repository into:

```
~/xv6-riscv
```

The RISC-V cross-compiler, the QEMU emulator and the other build tools were installed from the Ubuntu package repositories with:

```
sudo apt install -y git build-essential gdb-multiarch \
    qemu-system-misc gcc-riscv64-linux-gnu \
    binutils-riscv64-linux-gnu
```

The operating system was built and started using:

```
make qemu
```

After successful compilation, xv6 booted and displayed:

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

This confirmed that the xv6 kernel was running correctly and that the user shell had been started.

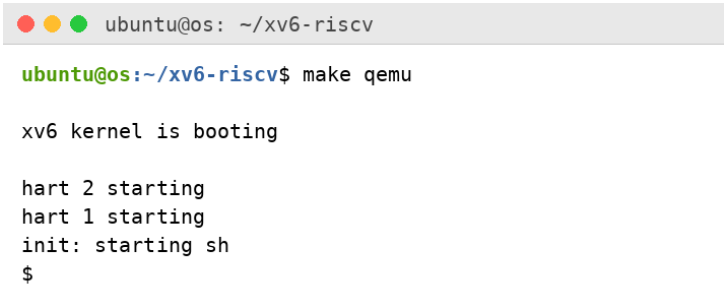


Figure 1. Building and booting xv6 with make qemu

3. Using xv6 as the Unix It Is

Several commands were executed inside the xv6 shell:

```
ls
echo hello xv6
ls | grep c
wc README
```

3.1. Observations

1. Three programs included with xv6 are:
   - `cat`
   - `echo`
   - `wc`
2. For a pipe such as `ls | grep c` to work, the operating system must support processes and inter-process communication. The shell runs the two commands as separate processes (created with fork and exec) and connects the standard output of the first process to the standard input of the second process using a pipe, which is a buffer kept in the kernel.
3. The xv6 shell is much simpler than a modern Linux shell such as Bash, but basic Unix commands and concepts such as ls, cat, echo, pipes, and standard input/output work in a similar way.

3.2. Example Command Output

The ls command displayed the programs available inside xv6 (the three columns are the file type, the inode number and the size in bytes):

```
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35376
echo           2 4 34312
forktest       2 5 17280
grep           2 6 42736
init           2 7 34768
kill           2 8 34240
ln             2 9 34032
ls             2 10 41520
mkdir          2 11 34280
rm             2 12 34264
sh             2 13 56136
stressfs       2 14 35160
usertests      2 15 200056
grind          2 16 50504
wc             2 17 36336
zombie         2 18 33648
logstress      2 19 36184
forphan        2 20 35048
dorphan        2 21 34488
sync           2 22 33712
console        3 23 0
```

The command echo hello xv6 produced:

```
hello xv6
```

The pipe ls | grep c kept only the lines of the listing that contain the letter c, and wc README counted 48 lines, 336 words and 2441 bytes in the README file:

```
cat             2 3 35376
echo            2 4 34312
wc              2 17 36336
sync            2 22 33712
console         3 23 0

48 336 2441 README
```

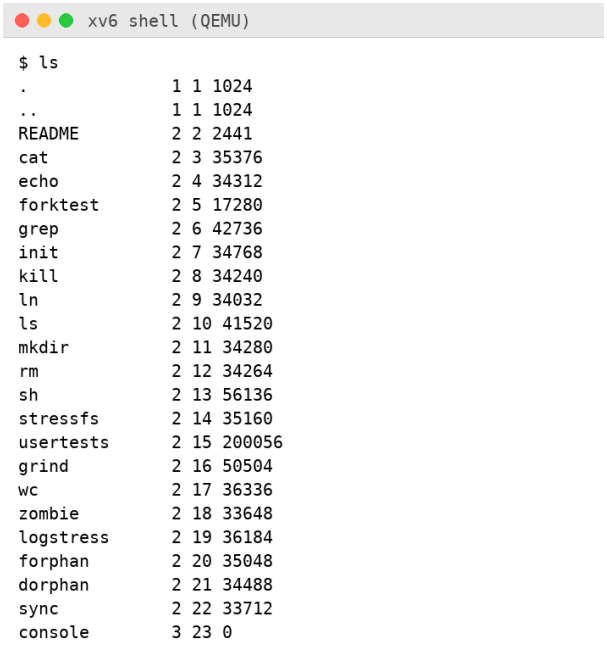


Figure 2. Programs available in the xv6 filesystem (ls)

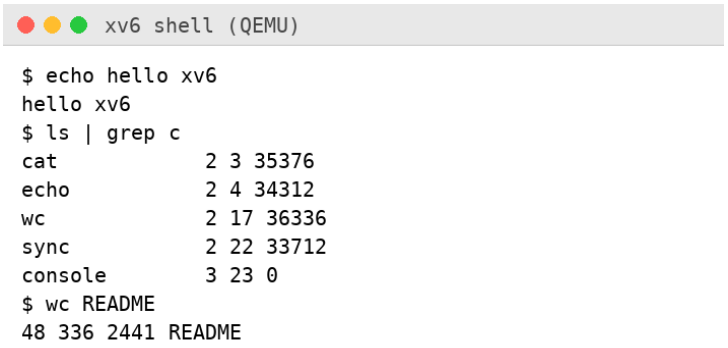


Figure 3. echo, a pipe with grep, and wc in the xv6 shell

4. Reading the Source

The source code of user programs and kernel system-call handlers was inspected using commands such as:

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c
```

4.1. System Calls Used by cat

The program user/cat.c uses several system calls.

- `read()` asks the kernel to read data from a file descriptor and place it into a buffer.
- `write()` asks the kernel to write data from a buffer to a file descriptor.
- `open()` asks the kernel to open a file.
- `close()` asks the kernel to close an opened file descriptor.
- `exit()` terminates the current process.

The main data-processing loop in cat is:

```
while ((n = read(fd, buf, sizeof(buf))) > 0) {
  if (write(1, buf, n) != n) {
    fprintf(2, "cat: write error\n");
    exit(1);
  }
}
```

The program repeatedly reads data from a file descriptor and writes that data to standard output (file descriptor 1).

4.2. Kernel Implementation

The implementation of sys_read was found in:

```
kernel/sysfile.c
```

The function starts at line:

```
69: sys_read(void)
```

The implementation of sys_write starts at line:

```
83: sys_write(void)
```

This demonstrates that a user-level function such as read() ultimately causes the operating system kernel to execute the corresponding system-call implementation.

4.3. Kernel and User Space

The directories kernel/ and user/ contain different types of code.

- `kernel/` contains privileged operating-system code that manages system resources and implements system calls.
- `user/` contains ordinary programs that run outside the kernel and request operating-system services through system calls.

This separation protects the kernel and system resources from direct access by user programs.

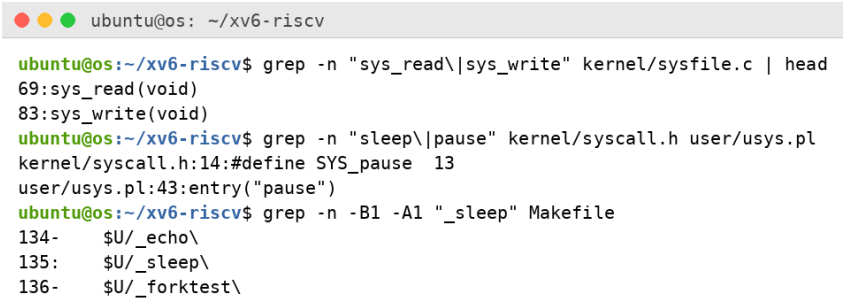


Figure 4. Locating sys_read and sys_write, the pause system call, and the Makefile entry

5. Creating a User Program

A new user program called sleep was added to xv6.

The source file was created as:

```
user/sleep.c
```

Its source code is:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));
  exit(0);
}
```

The program checks that exactly one argument was given, converts it from a string to a number with atoi, and asks the kernel to suspend the process for that number of clock ticks. In the version of xv6 used during this laboratory work, the system-call interface for suspending a process is named pause() (SYS_pause in kernel/syscall.h) instead of sleep().

The program was added to the UPROGS section of the Makefile:

```
$U/_sleep\
```

After rebuilding xv6 using:

```
make qemu
```

the new program became available inside the xv6 shell. Without the entry in UPROGS the build stops with the error that there is no rule to make target user/_sleep, and the program is not placed into the xv6 filesystem image.

5.1. Testing

The program was tested using:

```
sleep 10
```

The shell paused temporarily and then returned to the prompt. Calling the program without the required argument:

```
sleep
```

produced the expected error message:

```
usage: sleep <ticks>
```

This confirmed that the program was compiled, added to the xv6 filesystem, and executed successfully.

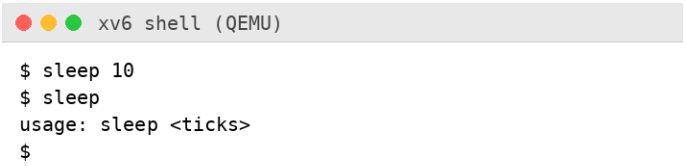


Figure 5. Testing the sleep program inside xv6

6. Conclusion

During this laboratory work, the xv6 operating system was successfully built and executed using the QEMU RISC-V emulator.

Basic Unix commands were used to explore the xv6 user environment, including commands involving pipes and standard input/output.

The source code of xv6 was then examined to understand the separation between user programs and kernel code and to observe how user programs access operating-system functionality through system calls.

Finally, a new user-level program, sleep, was created, added to the xv6 build system, compiled, and successfully executed.

The laboratory work provided a practical introduction to the internal structure of a Unix-like operating system and demonstrated the relationship between user programs, system calls, and kernel implementations.